Data Structures

BS (CS) _Fall_2025

Lab_06 Task

Learning Objectives:

1. Linked Lists (Doubly, Circular)

Lab Task 1: Student Grade Management System

Scenario

You are creating a grade management system for a university course. The system needs to maintain student records in alphabetical order by name (Insertion to be handled in a way that the list always remains in alphabetic order.

Task Requirements:

Create a doubly linked list where each node contains:

- Student ID (integer)
- Student Name (string)
- Grade (float, 0.0 to 100.0)
- Pointer to next student
- Pointer to previous student

Core Functionalities to Implement

1. **Student Management:**
   - addStudent(id, name, grade) - Insert student maintaining alphabetical order by name
   - removeStudent(id) - Remove student by ID
   - removeStudent(name) - Remove student by name
   - updateGrade(id, newGrade) - Update a student's grade
   - displayAllStudents() - Show all students in alphabetical order
2. **Search Operations:**
   - findStudentById(id) - Search and display student by ID
   - findStudentByName(name) - Search and display student by name
   - displayStudentWithNeighbors(id) - Show student with previous and next students
3. **Statistical Analysis:**
   - calculateClassAverage() - Compute average grade of all students
   - findHighestGrade() - Find and display student with highest grade
   - findLowestGrade() - Find and display student with lowest grade
4. **Implementation Guidelines**
   - Maintain alphabetical ordering during all insertion operations
   - Handle duplicate names by comparing IDs as secondary criteria

- Use proper memory management (constructor, destructor, copy constructor)

Test cases:

```
// 1. Add student (alphabetical order)
TEST(StudentListTest, AddAlphabetical) {
    StudentList s;
    s.addStudent(101, "Charlie", 85.0f);
    s.addStudent(102, "Alice", 90.0f);
    s.findStudentByName("Alice");   // should print Alice first
    s.findStudentByName("Charlie"); // should print Charlie second
}

// 2. Add duplicate name (compare by ID)
TEST(StudentListTest, AddDuplicateName) {
    StudentList s;
    s.addStudent(101, "Alice", 88.0f);
    s.addStudent(102, "Alice", 92.0f);
    s.displayAllStudents();  // should show both Alices
}

// 3. Remove student by ID
TEST(StudentListTest, RemoveById) {
    StudentList s;
    s.addStudent(101, "Bob", 80.0f);
    s.removeStudent(101);
    s.findStudentById(101); // should print not found
}

// 4. Remove student by Name
TEST(StudentListTest, RemoveByName) {
    StudentList s;
    s.addStudent(201, "David", 75.0f);
    s.removeStudent(std::string("David"));
    s.findStudentByName("David"); // should print not found
}

// 5. Update grade
TEST(StudentListTest, UpdateGrade) {
    StudentList s;
    s.addStudent(301, "Emma", 65.0f);
    s.updateGrade(301, 95.0f);
    s.findStudentById(301); // should show Emma with grade 95
}
```

```
// 6. Calculate class average
TEST(StudentListTest, ClassAverage) {
    StudentList s;
    s.addStudent(1, "A", 100.0f);
    s.addStudent(2, "B", 80.0f);
    EXPECT_FLOAT_EQ(s.calculateClassAverage(), 90.0f);
}

// 7. Find highest grade
TEST(StudentListTest, HighestGrade) {
    StudentList s;
    s.addStudent(1, "X", 50.0f);
    s.addStudent(2, "Y", 95.0f);
    s.findHighestGrade(); // should print Y with 95
}

// 8. Find lowest grade
TEST(StudentListTest, LowestGrade) {
    StudentList s;
    s.addStudent(1, "M", 30.0f);
    s.addStudent(2, "N", 75.0f);
    s.findLowestGrade(); // should print M with 30
}

// 9. Display student with neighbors (middle case)
TEST(StudentListTest, StudentWithNeighbors) {
    StudentList s;
    s.addStudent(1, "A", 70.0f);
    s.addStudent(2, "B", 80.0f);
    s.addStudent(3, "C", 90.0f);
    s.displayStudentWithNeighbors(2); // should show A (prev), B (current), C
(next)
}

// 10. Find student by ID (existent and non-existent)
TEST(StudentListTest, FindById) {
    StudentList s;
    s.addStudent(10, "Zara", 88.0f);
    s.findStudentById(10);   // should print Zara
    s.findStudentById(999);  // should print not found
}

// 11. Find student by Name (existent and non-existent)
TEST(StudentListTest, FindByName) {
    StudentList s;
```

```
    s.addStudent(11, "Liam", 77.0f);
    s.findStudentByName("Liam");   // should print Liam
    s.findStudentByName("Noah");   // should print not found
}

// 12. Remove non-existent ID
TEST(StudentListTest, RemoveNonExistentId) {
    StudentList s;
    s.addStudent(1, "Tom", 60.0f);
    s.removeStudent(999); // invalid
    s.findStudentById(1); // should still print Tom
}

// 13. Remove non-existent Name
TEST(StudentListTest, RemoveNonExistentName) {
    StudentList s;
    s.addStudent(2, "Sam", 55.0f);
    s.removeStudent(std::string("Unknown"));
    s.findStudentByName("Sam"); // should still print Sam
}

// 14. Single student case
TEST(StudentListTest, SingleStudent) {
    StudentList s;
    s.addStudent(100, "OnlyOne", 100.0f);
    s.findStudentById(100); // should print OnlyOne
    s.removeStudent(100);
    s.findStudentById(100); // should print not found
}

// 15. Multiple operations sequence
TEST(StudentListTest, MultipleOps) {
    StudentList s;
    s.addStudent(1, "Alpha", 70.0f);
    s.addStudent(2, "Beta", 80.0f);
    s.addStudent(3, "Gamma", 90.0f);
    s.removeStudent("Beta");
    s.updateGrade(3, 95.0f);
    s.displayAllStudents(); // should show Alpha (70) and Gamma (95)
}
```

Lab Task 2: Circular Linked List Implementation

Scenario

You are tasked with implementing a fundamental circular linked list data structure from scratch. This implementation will serve as the foundation for more complex circular data structures used in applications like round-robin scheduling, circular buffers, and continuous media playback systems.

Task Requirements

Implement a Node and CircularLinkedList Class

Core Functionalities to Implement

- CircularLinkedList() - Default constructor to initialize empty list
- insert(data) - Insert data item at the end of circular linked list
    - If list is empty, new node becomes head and points to itself
    - If list has elements, insert after the last node and maintain circularity
- isEmpty() - Check if the circular linked list is empty
- print() - Display all elements starting from head until reaching head again

Search and Update Operations:

- search(data) - Search for specific element, return true if found, false otherwise
- update(oldData, newData) - Replace/update one data element with another
- insertAtIndex(index, data) - Insert new element at specified position (0-indexed)
    - Count elements in the list to determine valid positions
    - Handle edge cases for index 0 and end positions

Deletion Operations:

- deleteData(data) - Delete the first occurrence of specified data element
- Handle special cases: single node, head node deletion, last node deletion

Test Cases:

```
// 1. Empty list
TEST(CircularListTest, IsEmptyInitially) {
    CircularLinkedList cl;
    EXPECT_TRUE(cl.isEmpty());
    EXPECT_EQ(cl.size(), 0);
}

// 2. Insert first element
TEST(CircularListTest, InsertFirstElement) {
    CircularLinkedList cl;
    cl.insert(10);
    EXPECT_FALSE(cl.isEmpty());
    EXPECT_EQ(cl.size(), 1);
    EXPECT_EQ(cl.getAtIndex(0), 10);
}

// 3. Insert multiple elements
TEST(CircularListTest, InsertMultipleElements) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    EXPECT_EQ(cl.size(), 3);
    EXPECT_EQ(cl.getAtIndex(2), 3);
}

// 4. Search existing element
TEST(CircularListTest, SearchElement) {
    CircularLinkedList cl;
    cl.insert(5);
    cl.insert(15);
    cl.insert(25);
    EXPECT_TRUE(cl.search(15));
    EXPECT_FALSE(cl.search(99));
}

// 5. Update element
TEST(CircularListTest, UpdateElement) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    EXPECT_TRUE(cl.update(20, 25));
    EXPECT_TRUE(cl.search(25));
```

```
    EXPECT_FALSE(cl.update(99, 100));
}

// 6. Insert at index 0 (head)
TEST(CircularListTest, InsertAtHead) {
    CircularLinkedList cl;
    cl.insert(5);
    cl.insert(10);
    cl.insertAtIndex(0, 99);
    EXPECT_EQ(cl.getAtIndex(0), 99);
}

// 7. Insert at middle
TEST(CircularListTest, InsertAtMiddle) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    cl.insertAtIndex(1, 50);
    EXPECT_EQ(cl.getAtIndex(1), 50);
    EXPECT_EQ(cl.size(), 4);
}

// 8. Insert at end
TEST(CircularListTest, InsertAtEnd) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    cl.insertAtIndex(2, 30);
    EXPECT_EQ(cl.getAtIndex(2), 30);
}

// 9. Insert invalid index
TEST(CircularListTest, InsertInvalidIndex) {
    CircularLinkedList cl;
    cl.insert(1);
    EXPECT_FALSE(cl.insertAtIndex(5, 99));
}

// 10. Delete head element
TEST(CircularListTest, DeleteHead) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    cl.insert(30);
```

```
    EXPECT_TRUE(cl.deleteData(10));
    EXPECT_FALSE(cl.search(10));
}

// 11. Delete last element
TEST(CircularListTest, DeleteLast) {
    CircularLinkedList cl;
    cl.insert(10);
    cl.insert(20);
    cl.insert(30);
    EXPECT_TRUE(cl.deleteData(30));
    EXPECT_EQ(cl.size(), 2);
}

// 12. Delete middle element
TEST(CircularListTest, DeleteMiddle) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
    EXPECT_TRUE(cl.deleteData(2));
    EXPECT_FALSE(cl.search(2));
}

// 13. Delete only element
TEST(CircularListTest, DeleteSingleNode) {
    CircularLinkedList cl;
    cl.insert(100);
    EXPECT_TRUE(cl.deleteData(100));
    EXPECT_TRUE(cl.isEmpty());
}

// 14. Delete non-existing element
TEST(CircularListTest, DeleteNonExisting) {
    CircularLinkedList cl;
    cl.insert(10);
    EXPECT_FALSE(cl.deleteData(999));
}

// 15. Print list (manual visual check)
TEST(CircularListTest, PrintList) {
    CircularLinkedList cl;
    cl.insert(1);
    cl.insert(2);
    cl.insert(3);
```

```
    cl.print(); // Just call (visual verification)
}
```